PlayArena — Sports Ground Booking and Management System

Kalvium Micro Project Report: Back-End Web Development (Modules 1, 2 & 3)

Student Name: Y Harsha Vardhan Reddy **Student Email:** yarasuharshavardhan.reddy.s.113@kalvium.community **Project Title:** PlayArena — Sports Ground Booking and Management System **Subject:** Back-End Web Development **Academic Year:** 2026 **Architecture:** Layered RESTful Express Architecture (Routes → Validators → Controllers → Services → Repositories → Prisma ORM → PostgreSQL) **Testing Framework:** Jest & Supertest

---

1. Executive Summary & Problem Formulation

Backend systems supporting recreational sports ground bookings require high data consistency, strict transactional boundaries, and robust concurrency handling. Conventional reservation systems frequently suffer from race conditions where two simultaneous API calls for the same time slot both succeed, causing scheduling collisions and customer dissatisfaction.

PlayArena Backend was built to provide an enterprise-grade RESTful API service that guarantees: - Complete defensive validation on all request payloads. - Clear separation of concerns across a 4-layer architecture. - Deterministic, atomic booking transactions backed by PostgreSQL partial unique indexes. - Consistent, predictable JSON contracts for success and error scenarios. - Comprehensive automated verification via 22 integration tests.

---

2. Subject Syllabus & Module Alignment

Module	Core Concepts Required	PlayArena Working Implementation
Module 1: Node.js & Express Foundations	Node.js runtime, Event loop, Express setup, Middleware chain, JSON body parsing, CORS policies, Environment configuration, Request tracking.	Modular server architecture in `server/src/app.js` and `server/src/server.js`; custom `requestIdMiddleware` attaching `req.id = crypto.randomUUID()` and `X-Request-Id` headers; `morgan` request logger; environment configuration using `dotenv`.

Module	Core Concepts Required	PlayArena Working Implementation
Module 2: RESTful API Design & Layered Architecture	Resource-oriented URL endpoints, Standard HTTP methods & status codes, Thin controllers, Service layer business logic, Repository isolation, Input validation.	REST API endpoints under `/api/v1/` for `auth`, `grounds`, `slots`, `bookings`, `analytics`, and `health`; Layered structure: thin controllers delegating to domain services; data access encapsulated in repositories; `express-validator` rules sanitizing inputs.
Module 3: Authentication, Security & Transactions	Password hashing with salt rounds, JWT issuance & verification, Role-Based Access Control (RBAC), ACID transactions, Concurrency conflict resolution, Centralized error handling.	BCrypt password hashing (10 salt rounds); JWT token lifecycle management (`jsonwebtoken`); `authenticate` and `authorize('ADMIN')` middleware; Prisma interactive transactions (`prisma.$transaction`) enforcing atomic booking and slot state changes; translation of PostgreSQL uniqueness violations into HTTP 409 Conflict responses.

---

3. Backend Layered Architecture

```
server/
├── prisma/
│   ├── schema.prisma           # Normalized PostgreSQL relational schema
│   ├── seed.js                 # Database seeder (Admin, Users, 6 Grounds, 240 Slots)
│   └── migrations/             # Version-controlled SQL migrations & constraints
├── src/
│   ├── app.js                  # Express application setup & middleware assembly
│   ├── server.js               # HTTP server listener & graceful shutdown handlers
│   ├── config/                 # Environment variables & singleton Prisma instance
│   ├── errors/                 # Custom application error classes (AppError, ConflictError, e
│   ├── utils/                  # Response formatters, bcrypt password helpers, JWT signing
│   ├── middleware/             # requestId, requestLogger, auth, validate, errorHandler, notF
│   ├── validators/             # express-validator schemas for auth, grounds, slots, bookings
│   ├── repositories/           # Isolated database access objects (User, Ground, Slot, Bookir
│   ├── services/               # Domain logic, transaction boundaries, trusted pricing
│   ├── controllers/            # Request parsing, service invocation, standardized responses
│   └── routes/                 # Express route definitions grouped by resource
└── tests/
    ├── auth.test.js            # Registration, login, profile, credential validation tests
    ├── grounds.test.js         # Ground listing, filtering, search, and RBAC creation tests
    ├── booking.test.js         # Booking lifecycle, trusted price, cancellation tests
    └── concurrency.test.js     # Competing parallel booking race condition tests
```

---

4. RESTful API Response Contracts

4.1 Success Response Contract

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "total": 6,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

4.2 Error Response Contract

```
{
  "success": false,
  "error": {
    "code": "SLOT_ALREADY_BOOKED",
    "message": "This time slot is no longer available. Another booking has already reserved it.
  }
}
```

---

5. Critical Service Logic: Atomic Booking & Concurrency Protection

In `server/src/services/booking.service.js`, the booking creation workflow executes within an isolated transaction boundary:

```
async createBooking({ userId, slotId, notes }) {
  const parsedSlotId = parseInt(slotId, 10);

return prisma.$transaction(async (tx) => {
    // 1. Fetch slot with existing active bookings inside transaction
    const slot = await tx.slot.findUnique({
      where: { id: parsedSlotId },
      include: {
        ground: true,
        bookings: { where: { status: { in: ['HELD', 'CONFIRMED'] } } }
```

```
      }
    });


if (!slot) throw new NotFoundError('Time slot not found.');
    if (!slot.isAvailable || (slot.bookings && slot.bookings.length > 0)) {
      throw new SlotAlreadyBookedError('This time slot is no longer available.');
    }


// 2. Trusted backend price calculation (Never trust client body)
    const calculatedTotal = slot.price;


// 3. Generate unique booking reference
    const bookingReference = PA-${new Date().toISOString().slice(0, 10).replace(/-/g, '')}-${Ma


// 4. Atomic Booking Insert
    // Protected by PostgreSQL partial unique index 'unique_active_booking_per_slot'
    const createdBooking = await tx.booking.create({
      data: {
        bookingReference,
        userId,
        groundId: slot.groundId,
        slotId: slot.id,
        totalAmount: calculatedTotal,
        status: 'CONFIRMED',
        notes
      }
    });


// 5. Update slot availability flag
    await tx.slot.update({
      where: { id: slot.id },
      data: { isAvailable: false }
    });


return createdBooking;
  });
}
```

---

6. Automated Testing Suite & Concurrency Results

The backend includes 4 dedicated integration test suites running under Jest and Supertest against the live PostgreSQL database.

6.1 Test Execution Summary

```
Test Suites: 4 passed, 4 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        3.546 s
Ran all test suites.
```

6.2 Detailed Test Cases Matrix

Test Suite	Test Case	Target Behavior	Observed Result	Status
`auth.test.js`	User Registration	Valid payload creates user and issues JWT	HTTP 201 Created with valid token	**PASS**
`auth.test.js`	Duplicate Email Rejection	Attempt to register existing email	HTTP 409 Conflict (`EMAIL_ALREADY_EXISTS`)	**PASS**
`auth.test.js`	Malformed Email Rejection	Non-email format in email field	HTTP 400 Bad Request (`VALIDATION_ERROR`)	**PASS**
`auth.test.js`	Successful Login	Correct email & password	HTTP 200 OK with JWT token	**PASS**
`auth.test.js`	Invalid Password Login	Incorrect password	HTTP 401 Unauthorized (`INVALID_CREDENTIALS`)	**PASS**
`auth.test.js`	Profile Retrieval (`/me`)	Valid Bearer token	HTTP 200 OK with authenticated user profile	**PASS**
`auth.test.js`	Unauthenticated Profile Request	Missing Bearer token	HTTP 401 Unauthorized	**PASS**
`grounds.test.js`	Paginated Grounds List	Fetch grounds with page & limit	HTTP 200 with data array and pagination metadata	**PASS**
`grounds.test.js`	Sport Type Filtering	Filter by `sportType=Football`	All returned items have `sportType: 'Football'`	**PASS**
`grounds.test.js`	Search Query Matching	Query `search=Apex`	Correct ground returned matching keyword	**PASS**
`grounds.test.js`	Single Ground Details	Fetch ground by existing ID	HTTP 200 with details and facilities array	**PASS**

Test Suite	Test Case	Target Behavior	Observed Result	Status
`grounds.test.js`	Missing Ground 404	Query non-existent ID `999999`	HTTP 404 Not Found (`GROUND_NOT_FOUND`)	**PASS**
`grounds.test.js`	Ground Slots by Date	Fetch slots for ground by date	Array of slots with start/end times and prices	**PASS**
`grounds.test.js`	Admin Create Ground	Admin token creating new venue	HTTP 201 Created with persisted record	**PASS**
`grounds.test.js`	Non-Admin Ground Creation	Regular user token creating venue	HTTP 403 Forbidden (`FORBIDDEN`)	**PASS**
`booking.test.js`	Create Booking	Valid slot ID from authenticated user	HTTP 201 Created with booking reference	**PASS**
`booking.test.js`	Trusted Price Calculation	Verify `totalAmount` matches slot price	Database price applied, ignoring client body	**PASS**
`booking.test.js`	Reject Occupied Slot	Booking attempt on occupied slot	HTTP 409 Conflict (`SLOT_ALREADY_BOOKED`)	**PASS**
`booking.test.js`	User Booking History	List bookings for current user	Returns only reservations owned by user	**PASS**
`booking.test.js`	Unauthorized Cancellation	User 2 attempting to cancel User 1 booking	HTTP 403 Forbidden (`NOT_BOOKING_OWNER`)	**PASS**
`booking.test.js`	Successful Cancellation	Booking owner cancelling reservation	Status set to CANCELLED; slot marked available	**PASS**
`booking.test.js`	Repeated Cancellation Rejection	Cancelling already cancelled booking	HTTP 400 Bad Request (`ALREADY_CANCELLED`)	**PASS**
`concurrency.test.js`	Parallel Competing Requests	`Promise.all` 2 competing booking calls for same slot	Exactly ONE 201 Created, exactly ONE 409 Conflict	**PASS**

6.3 Concurrency Race Condition Test Evidence

In `tests/concurrency.test.js`, two concurrent HTTP POST requests were dispatched simultaneously via `Promise.all`:

```
⚡ Launching 2 simultaneous competing booking requests for Slot ID: 13
Unique constraint failed on the fields: (slotId)
```

```
📊 Concurrency Test Result Statuses: [409, 201]
```

PostgreSQL's engine-level partial unique index triggered a unique constraint violation on the second request, which our centralized error middleware cleanly transformed into an HTTP 409 Conflict with error code `SLOT_ALREADY_BOOKED`. Direct database inspection confirmed that exactly one active booking was persisted.

---

7. Reliability & Defensive Engineering

- **Graceful Shutdown:** `SIGINT` and `SIGTERM` signals trigger HTTP server closure followed by safe disconnection of the Prisma Client connection pool. - **Defense in Depth:** Request data is validated by express-validator, verified by service business logic, and guarded by database check constraints and partial indexes. - **Information Leakage Prevention:** Stack traces and internal database error objects are stripped from production API responses.

---

8. Conclusion

The PlayArena backend provides a textbook demonstration of Back-End Web Development Modules 1, 2, and 3. Through modular routes, layered architecture, strict JWT/RBAC security, transaction boundaries, and verified concurrency handling, the backend delivers exceptional stability and reliability.